

Lab 3: Component Modelling & Architectural Pattern Selection

Problem Statement: Internal Microservice Catalog & Health Portal

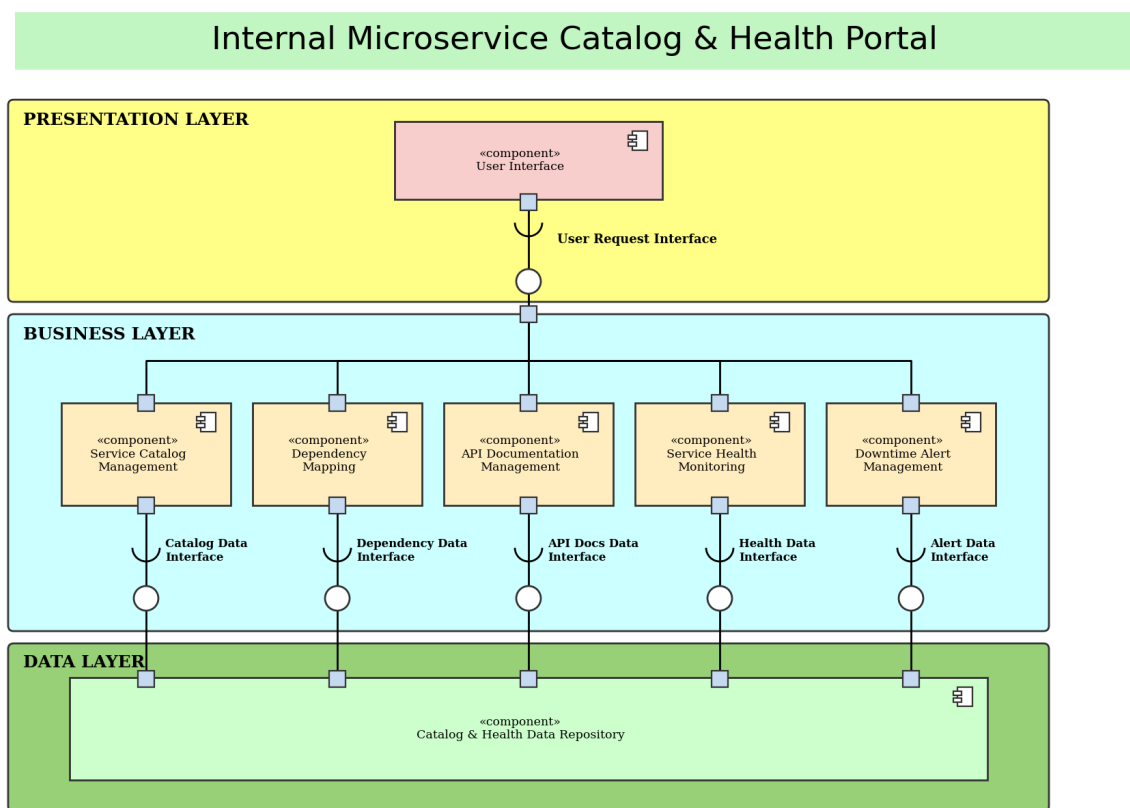
Architecture: Layered Architecture

Name: Kolluri Sruthi

Section: D

SRN: PES1UG24CS230

Step:1



Architecture Selection

We chose **Layered Architecture** for the Internal Microservice Catalog & Health Portal. The system is divided into three layers. The Presentation Layer contains the User Interface. The Business Layer contains Service Catalog Management, Dependency Mapping, API Documentation Management, Service Health Monitoring, and Downtime Alert Management. The Data Layer contains the Catalog & Health Data Repository. Each layer communicates only with the layer directly below it, through defined interfaces.

Reason 1 – Separation of Concerns

The portal combines several different responsibilities: displaying dependency graphs and API documentation, running 30-second health probes, calculating rolling 24-hour availability, and dispatching downtime alerts. The layered structure keeps the user interface, the business rules, and the data storage separate. Each business component (for example, Service Health Monitoring) can be developed, tested, and modified without affecting the others or the user interface.

Reason 2 – Maintainability and Ease of Change

The business components communicate with the data layer only through defined interfaces (Catalog Data, Dependency Data, API Docs Data, Health Data, and Alert Data Interfaces). The repository technology or the dependency-graph display can therefore be changed without rewriting the business logic. This suits an internal tool that will evolve as new services and requirements are added.

Security Advantage

The User Interface never accesses the data layer directly. Every request passes through the User Request Interface and the business layer, so authentication and authorization (NFR-002) can be enforced at one controlled entry point. Only authenticated and authorized users can reach the catalog and health information, and the data layer is not exposed to users at all.

Performance Benefit

Because the layers are separate, each can be optimized on its own. The Service Health Monitoring component can run the 30-second probes (FR-001) and store the results independently of the user interface. The data layer can be indexed and tuned for availability calculations. The dependency graph viewer can be optimized on its own so that graphs of up to 200 services render smoothly (NFR-001).